



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – ВАРНА

Факултет по изчислителна техника и автоматизация

Катедра „КНТ“

Документация

по дисциплината „Избрани методи за машинно обучение“

на тема: „Monte Carlo методи“

Изготвил: Мартин Чалъков 23621933

Специалност: Изкуствен Интелект

Проверил: х.ас. П. Димитрова

Съдържание:

1. Увод
2. Исторически контекст и основна идея на метода
3. Технически детайли и различни видове Monte Carlo методи
4. Реализирани примери
5. Предимства / недостатъци
6. Използвана литература

1. Увод

Monte Carlo методите представляват клас числени алгоритми, които използват случайни числа и редица вероятностни модели за да решават различен набор задачи. Намират приложение при разработки, които изискват прекалено много изчисления, нереалистични за продължително тестване и трудни за анализ, или които нямат точно аналитично решение. Основната цел на метода се постига с помощта на редица случайни експерименти върху входните данни, чрез които се получава общ приблизителен резултат. Увеличаването на броя опити води до по-близък и точен отговор.

В съвременните технологии Monte Carlo методите намират широко приложение в сфери като машинно обучение, икономически и бизнес насочени разработки, физически, инженерни и компютърни науки. Методите играят важна роля в моделирането в създаването и подобряването на сложни системи, предвидени за анализ и откриване на несигурности.

2. Исторически контекст и основна идея на метода

Днешното разбиране и същността на Monte Carlo методите е благодарност на Станислав Улав през 40-те години на XX век. В следствие на тежка операция на мозъка, която води до временния края на кариерата му като преподавател, той започва да анализира и решава различни математически задачи, една от които е свързана с игра на Пасианс. В него се пробужда въпроса какви са шансовете една ръка да бъде с всички подредени 52 карти. Такова изчисление би отнело много време и математически усилия, тъй като един дек има 8×10^{67} начина да се подреди. Следователно стига до заключението, че ако направят 100 опита с размесено тесте и случайни ръце, може да се изчисли желаната вероятност.

Тази негова идея намира приложение след като се завръща в Лос Аламос лабораторията, където теорията и изчисленията биват приложени за редица проблеми с неутронна дифузия, математическа физика и разработването на водородната бомба. В комбинирани усилия с математик и физик Джон фон Нойман в рамките на Манхатанския проект, те разработват първите методи, създадени за първите електронни компютри. В процес на работа с тях, методите получават кодовото име „*Monte Carlo*“.

В последствие на тези техни открития и тяхното публикуване, Monte Carlo методите бързо набират популярност и намират широко приложение в редица сфери в които може да се използват компютърни изчисления. Основната идея и метод на работа включва:

- Генерирането на случайни стойност;
- Извършване на множество симулации (още наричани епизоди);
- Статистическа обработка и сумиране на резултатите на симулациите.

3. Технически детайли и различни видове Monte Carlo методи.

В основата на Monte Carlo методите стоят принципите на теорията на вероятностите и статистиката. Предполага се, че при достатъчно голям брой случайни стойности може да се доближим до очакваната стойност. Това се базира на следната формула:

$$E[f(x)] = \int f(x)p(x)dx \approx \frac{1}{n} \sum_i f(x_i)$$

Основните компоненти на формулата са:

- $f(x)$ - функцията която изследваме;
- $p(x)$ – вероятностно разпределение (добавя се в случай на неравномерно разпределени проби);
- X_i – случайни проби, генерирани от избраното разпределение;
- n – брой симулации, колкото е по-голяма стойността толкова по-точен е резултата;
- $1/n \sum f(x_i)$ – апроксимация, представлява крайната средна стойност на всички резултати.

Имайки това в предвид, може да се обясни как методите изпълняват основната си цел - доближат се до реален резултат с помощта на ново създадени случайни проби. Всеки отделен вид метод има различен начин на постигане на тази цел, но цялостно погледнато методите следват сходна логика:

- Дефинира се домейн от входни данни;
- Генерират се нови данни на база вече съществуващите;
- Извършват се множество симулации, като всяка нова симулация подобрява резултатите на следващите;
- Резултатите се обработват чрез статистически методи, избрани според дадената цел;
- Създава се апроксимация на търсената стойност;
- Прави се оценка и анализ на резултатите;

Важно е да се подчертае, че различните Monte Carlo методи използват различен вид проби – зависими и независими. Базовия Monte Carlo метод използва независими проби, докато по-сложните (като *МСМС*) използват зависими проби, тъй като всяка следваща нова стойност се определя на база на предишната. Това си има своите плюсове, но може да доведе до автокорелация.

Различните Monte Carlo методи се отличават един от друг по начина на създаване/подбиране на нови данни. Всеки метод представя силни и слаби страни в зависимост от желания резултат и сферата в която ще бъде използван. Най-често

срещаните разновидности на Monte Carlo метода са Markov Chain Monte Carlo, Rejection Sampling и Importance Sampling.

- **Markov Chain Monte Carlo (MCMC)** – взема решения базирано на Марковското свойство, което работи на следния принцип: започвайки от началната стойност, метода генерира нова стойност кандидат на база началната. След това се изчислява вероятността с която новата стойност ще се приема или не, след което при взетото решение се повтаря процеса до стигане на желан брой нови стойности. Прилагането на зависим кандидат стойности води до по-бързо изследване на пространството и създаването на по-стабилни решения и стойности които са възможно най-близки до реалните, но може да доведе и до автокорелация.

Предимство на *MCMC* е високата му скорост на обхождане на пространството и *burn-in* периода от реализацията му – този период игнорира началните стойности във веригата, тъй като те не представят добре остатъка от разпределението. Това, комбинирано с *mixing* се прави с цел избягването на автокорелация между последователните стойности, което води до спад в продуктивността и крайните резултати.

MCMC се прилагат в невронни мрежи и редица статистически модели като Бейсова статистика. Недостатък на *MCMC* е зависимостта му от началната точка при създаване на кандидатите. В случай където началните стойности не са подбрани добре/разпределението е неправилно, крайните резултати ще бъдат неточни и неясни.

- **Importance Sampling** (извадки по важност) – представлява метод на оценка на свойствата на дадено разпределение, като се изпозват извадки които са генерирани извън интересувашото ни разпределение. Процеса се повтаря до събирането на нужния брой стойности. Основна силна черта е разнообразието при избирането на извадките, което се постига чрез задаване на тежести на всяка нова стойност. Това води до по-точна оценка и помага да се компенсира разликата между реалното и използваното разпределение. Важен плюс за Importance Sampling метода, е че се справя добре с пространства с шум в тях. Намира приложение в Бейсови модели, но намира и честа употреба в оценки на вероятностите както и подсилено машинно обучение.
- **Rejection Sampling** (отхвърляне на извадки) – представлява метод за генериране на случайни стойности от сложно (целево) разпределение, чрез използване на по-просто помощно разпределение като основа. Генерират *кандидат* стойности от помощното разпределение, които след това се проверяват дали да се приемат/отхвърлят въз основа на зададено условие, определено с плътностна функция на целевото разпределение. Процеса се повтаря докато не се съберат

достатъчен брой валидни стойности, които следват желаното от нас разпределение.

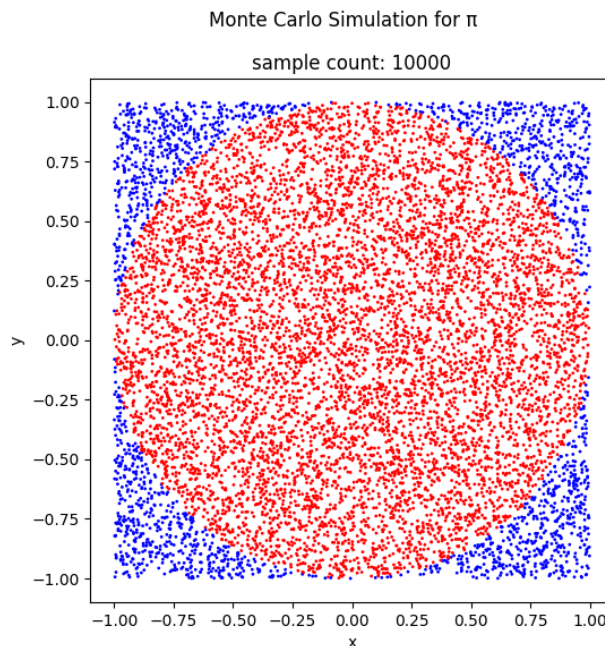
Затруднение при използването на метода се среща при ситуации където генерираните стойности биват повторно отхвърлени. Тази трудност се среща при неправилно избиране на помощното разпределение. Ако то не покрива добре целевото разпределение, голяма част от генерираните стойности ще бъдат отхвърлени, което следователно води до ниска ефективност и увеличено време за изчисление.

След генерирането на случайните проби, следващата ключова стъпка при използването на Monte Carlo методите е обработката и проверка на крайните резултати. Това се прави с цел намаляне на грешките и разминаването в крайните резултати. Възможни причини за срещане на грешки или неправилно обучение представляват случайността в данните и подбирането им (води до стохастични грешки), лош избор на разпределение, автокорелация (среща се при *МСМС*). С цел избягването на тези грешки, в тази стъпка от процеса на методите случайните стойности биват обработени и превърнати в надеждни резултати, които са максимално сходни до реалния резултат. Основните подходи за постигане на тази цел включват:

- **Апроксимация:** Най-често използвания (и най-простия) подход е чрез изчисляване на средна стойност на всички получени резултати;
- **Сходимост на резултатите:** с увеличаването на броя симулации, резултатите започват да се стабилизират в определен момент. Това позволява да се оцени кога е била постигната достатъчна точност;
- **Доверителни интервали:** стабилизират резултатите като се увеличи броя на симулациите;
- **Намаляване на вариацията:** прилагат се техники като импоранс самплинг за да се намали разсейването на резултатите и да се подобри точността, без да се налага значително увеличаване на броя симулации.

4. Реализирани примери

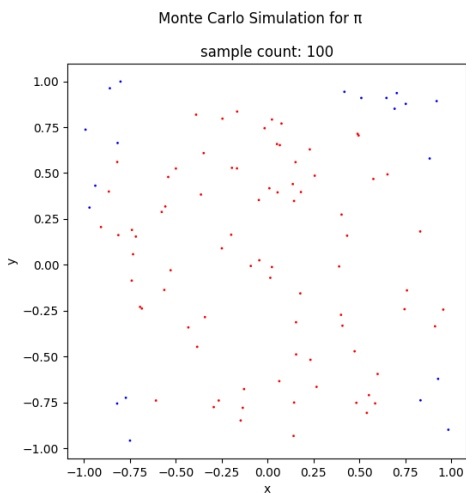
- **Основен Monte Carlo метод:** апроксимация на стойността на числото π . Използват се случайни проби в дадения квадрат, като се проверява дали те попадат в окръжност. Чрез съотношението между отделните точки се получава апроксимация на π .



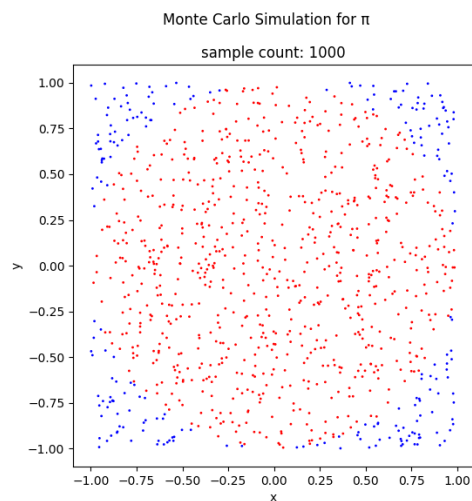
Estimated value of π with 10000 samples: 3.144

Важно е да се отбележи, че колкото повече са новите случайните стойности, толкова повече резултата се доближава до реалната стойност на π .

Тук може да се види разликата между използването на 100 или 1000 случайни нови стойности, в сравнение с 10000 от предишния пример:



Estimated value of π with 100 samples: 3.32



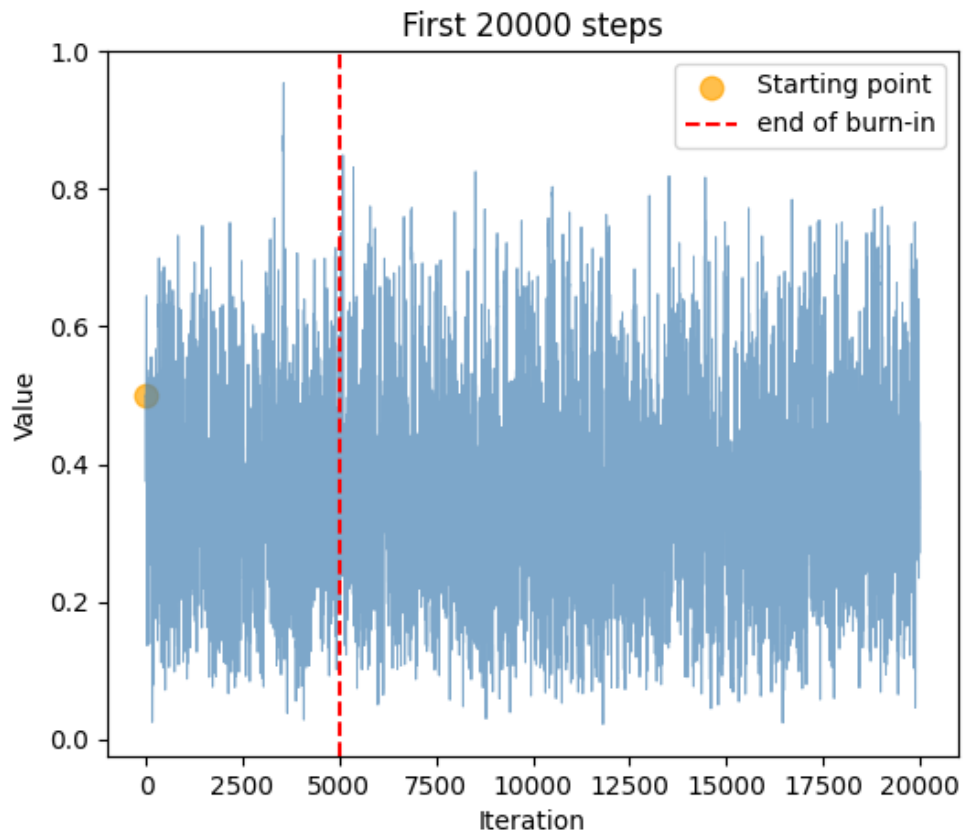
Estimated value of π with 1000 samples: 3.116

От реализираните два примера с по-малък брой стойности, може да се стигне до заключението, че за да се обучават правилно, Monte Carlo методите се нуждаят от сравнително голям брой входни данни. Както се забележи с показаните примери, нисък брой на входни води до големи неточности и объркване.

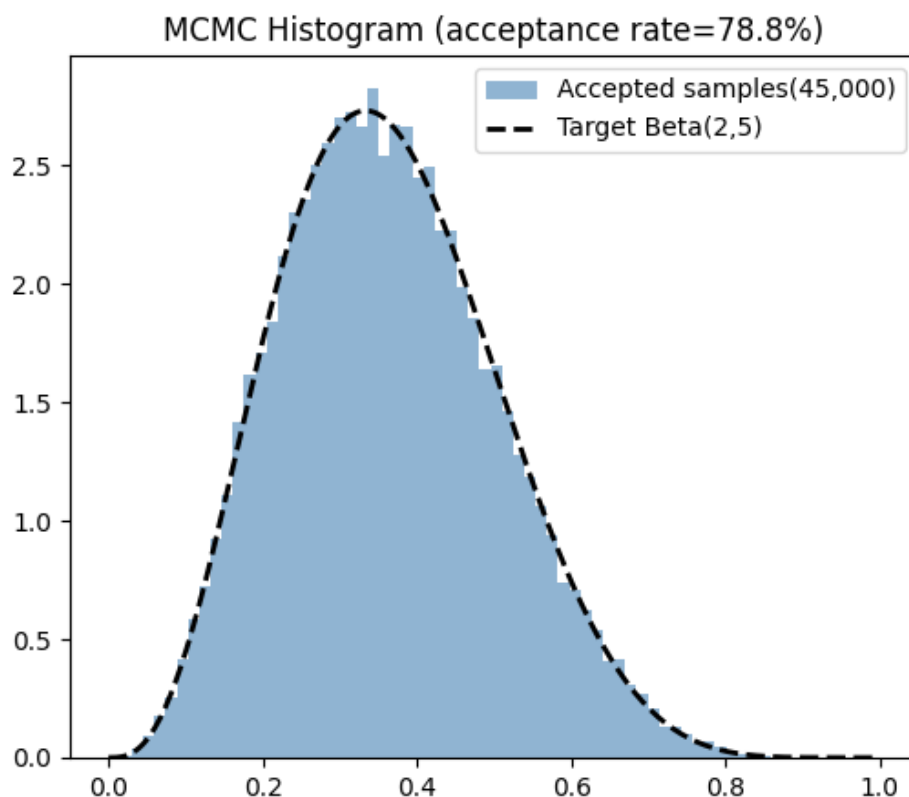
- **Markov Chain Monte Carlo, Metropolis-Hastings:** Взимането на кандидат стойности от Beta(2,5), но вместо да взимаме директно от него се строи верига от стъпки, като всяка стъпка зависи от предишната. След определен брой стъпки, веригата започва да взима стойности с честота която съответства на целевото разпределение.

За целта са избрани 50000 стъпки за веригата, като всяка от тях е с отскок 0.1 една от друга.

За първите 5000 точки се избира burn in период. Прави се с цел избягване веригата да зависи прекалено от тях в случай че данните са небалансирани



На графиката са изобразени първите 20000 точки и последователността която Марковската верига следва. Първите 5000 стъпки се вижда голяма несигурност в стойностите, но след това започва да се следва постоянно ниво на изследване на пространството. Може да се види и избраната от нас начална точка.

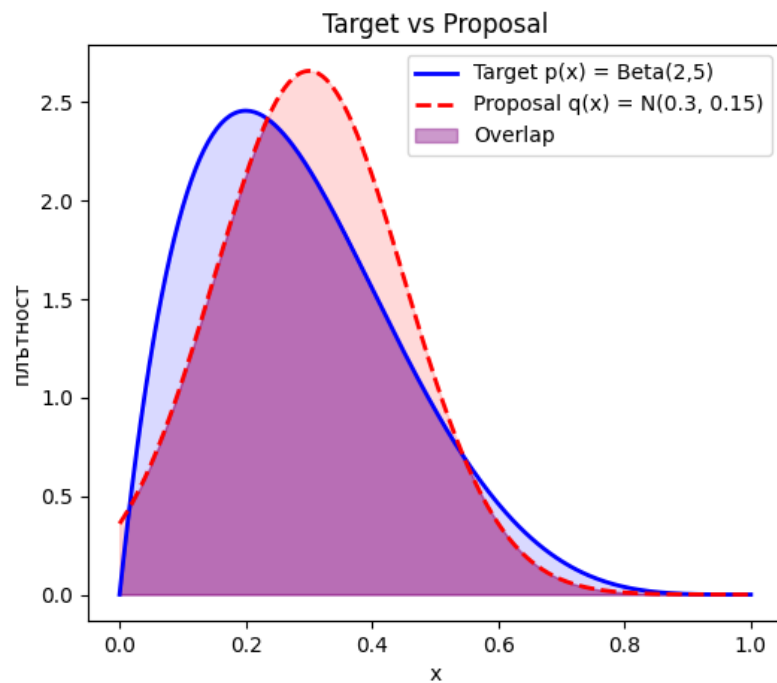


Хистограмата показва че генерираните стойности съвпадат максимално много с избраната Beta функция. Може да се установи, че алгоритъма се справя добре и успешно обхожда разпределението и апроксимира целевото разпределение.

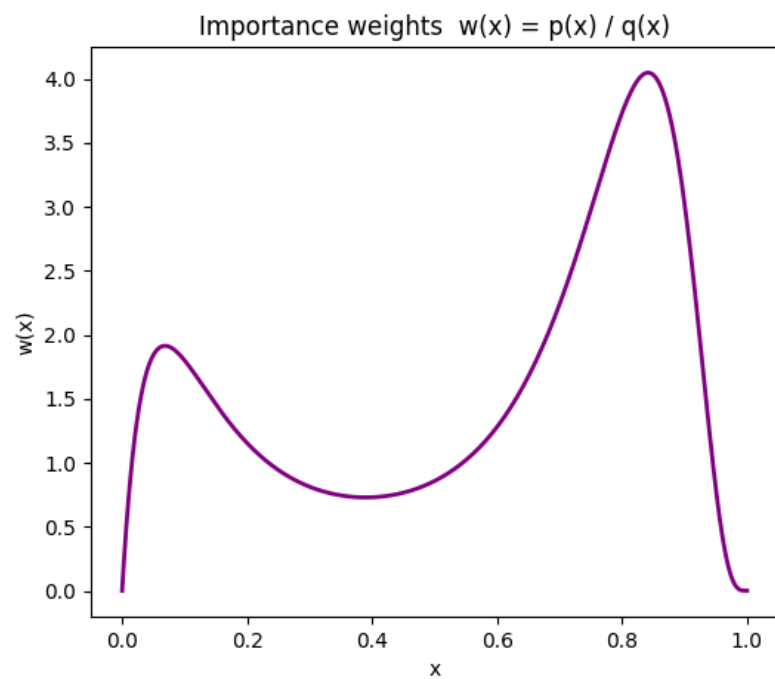
Acceptance rate: 78.8%

Вижда се че има 78% приемане, което показва че модела е стабилен.

- **Importance Sampling:** Изчисляване на очаквана стойност чрез генериране на случайни стойности от помощно разпределение.
 Примера разглежда две разпределения: *target* и *proposal*. *Target* е разпределението което искаме да получим накрая с избрана функция $\text{beta}(2,5)$, *proposal*(0.3, 0.15) е предложено помощно разпределение от което се взимат пробите. В него се генерират 20000 случайни стойности.
 Зададено е ограничение върху възможните стойности да е между 0 и 1.
 Добавят се тежести към стойностите от *proposal* разпределението с цел компенсиране на разликата между двете разпределение. Ако определена зона от разпределението бива подценена, точките в нея получават по-голямо тегло. Вероятностите се нормализират във вероятности.



На графиката са изобразени *target* и *proposal* разпределенията, може да се види къде се получава препокриване между двете разпределения. Голямо препокриване води до по-точна и стабилна оценка.

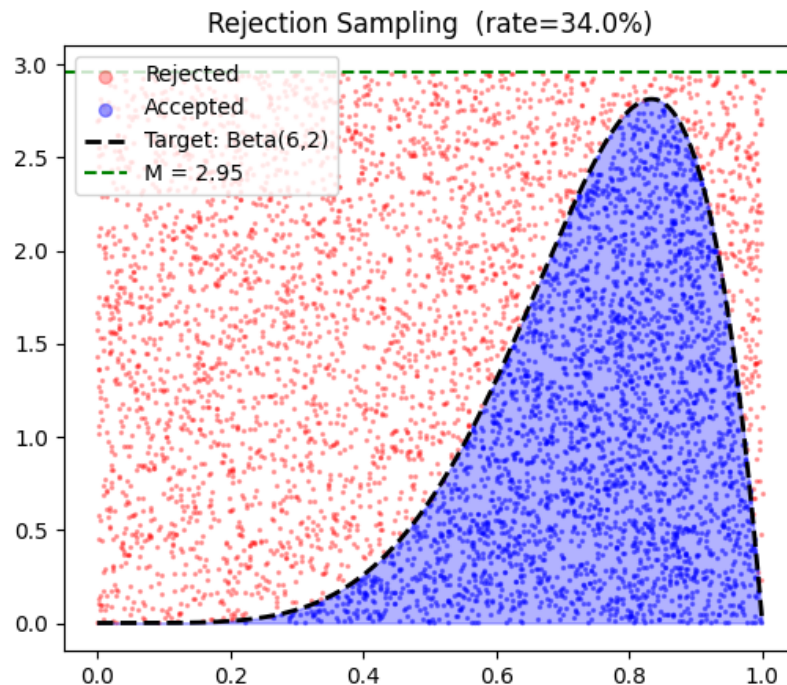


Графиката показва на кои стойности се дава повече тегло в сравнение с други. Установяваме, че стойностите които са извън съпадащото пространство имат по-висока тежест.

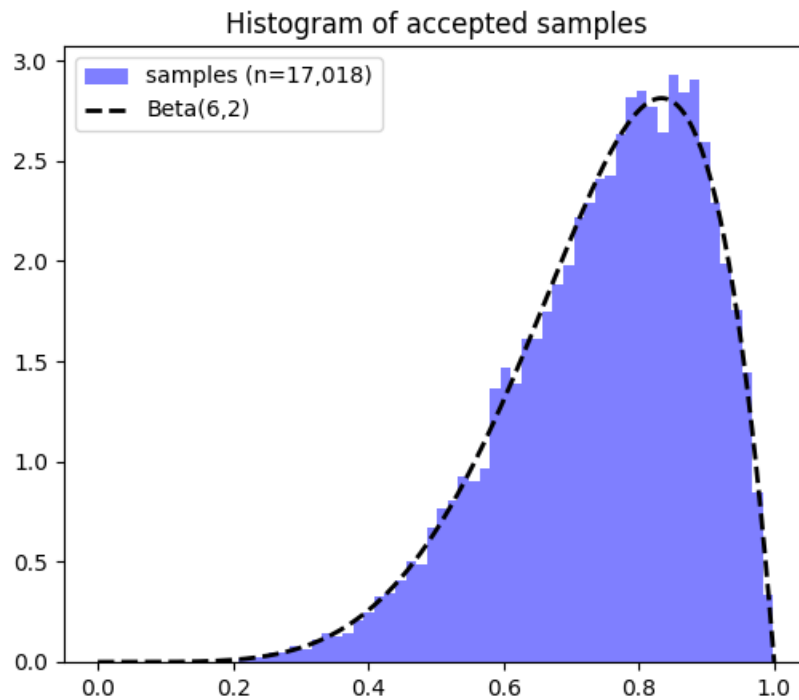
```
Cumulative score from Normal= [0.14025559 0.10664042 0.12271475 ... 0.10815944 0.10817007 0.10816015]  
Exact score of Beta(2,5) = 0.10714285714285714
```

Cumulative_score и *exact_score* служат за сравнение на *Beta* и получения резултат. Може да се установи че модела се е справил добре и се е доближил максимално до желаната стойност.

- **Rejection Sampling:** Визуализиране на вероятността точка в пространството да бъде приета или отхвърлена, спрямо зададено целево разпределение. Създават се 50000 случайни точки, като кандидат стойностите се генерират от равномерно разпределение в интервал $[0,1]$. То работи като помощно разпределение от което ще се генерират нови кандидат стойности, които ще бъдат приети или отхвърлени.
За целево разпределение е избрана функцията *Beta* с параметри $a=6$ и $b=2$ с която ще се определи формата на вероятностната крива. Избрана е горна градина M , която представлява максималната стойност която *Beta* може да достигне. Умножена е по малък буфер ($\times 1.05$) за да се гарантира, че желаното разпределение ще покрие напълно целевото.



От реализираната графика може да се определи дали случайно генерираните стойности попадат под стойността на функцията. Ако за дадена точка случайно генерираната стойност е по-малка от стойността на функцията, тя се приема (маркирана в синьо), ако е по-голяма от стойността на функцията се отхвърля (маркирана в червено).



На хистограмата се визуализира разпределението и броя на приетите стойности. Вижда се, че те следват максимално формата на $Beta(6,2)$, което показва че методът правилно успява да генерира проби от целевото разпределение.

Acceptance rate: 33.9%

Забелязва се 33.9% на приемането, което показва че $\sim 34\%$ от генерираните точки са били приети. От конкретния пример може да се установи че приблизително една трета от стойностите в разпределението са попаднали под плътностната функция и са били използвани за изграждане на крайното разпределение. Тази стойност силно зависи от избраната функция и площта която тя обхваща от разпределението.

5. Предимства и недостатъци на Monte Carlo методите

Предимства:

- Лесни за имплементация, и прости от страна на алгоритъм и концептуална реализация, тъй като не искат сложни математически преобразувания;
- Приложими са в различни области и предлагат широк избор от различни алгоритми за нужната сфера/задача;
- Позволяват промяна и адаптиране към различни проблеми и модели;
- Възможно е осъществяването на паралелизация;
- Позволяват решение на задачи с висока размерност, при които други аналитични методи не биха се справили;
- Разновидности като Importance Sampling се справят добре с пространства от данни които са шумни или несигурни;

Недостатъци:

- Лоши входни данни и/или неправилно разпределение (включително и помощно ако използваме) водят до неточни резултати;
- Ако новосъздадените *кандидат* стойности, както и предишните разпределения на които са базирани, са неравномерно разпределен, крайните резултати ще бъдат неточни;
- Висок брой и интензивност на изчисления. Методите спестяват човешките усилия, но изискват голяма компютърна мощ;
- Въпреки високата си успеваемост, Monte Carlo методите очакат по-голям набор от входни данни за да успеят правилно да извършат изчисленията;
- Възможно е да има различни резултати при повторно тестване, въпреки че данните остават същите. Това се дължи на факта, че различните епизоди имат различно трениране, има случаи където пропуска възможности ако не е добре запознат/няма достатъчно входни данни.
- Възможно е възникване на автокорелация между пробите (най-често срещано в *МСМС*), което намаля качеството на резултатите;
- Резултатите са вероятностни, което може да направи анализа им труден в някои случаи.

Въпреки изброените недостатъци, Monte Carlo методите са едно от най-революционните открития в сферата на компютърните изчисления и остават един от най-ефективните и широко използвани инструменти за решаване на стохастични задачи. Тяхната гъвкавост и приложимост ги правят важна част от съвременните науки и технологии.

6. Използвана литература

- Hitting the Jackpot: The Birth of the Monte Carlo Method - <https://www.lanl.gov/media/publications/actinide-research-quarterly/1123-hitting-the-jackpot-the-birth-of-the-monte-carlo-method>
- Monte Carlo method - https://en.wikipedia.org/wiki/Monte_Carlo_method
- Monte Carlo Methods for Reinforcement Learning - https://www.tutorialspoint.com/machine_learning/machine_learning_monte_carlo_methods.htm
- Monte Carlo Methods - <https://people.duke.edu/~ccc14/sta-663/MonteCarlo.html>
- Markov chain Monte Carlo - https://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo
- Markov chain Monte Carlo (MCMC) - <https://www.geeksforgeeks.org/artificial-intelligence/markov-chain-monte-carlo-mcmc/>
- Markov chain Monte Carlo - https://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo
- Importance sampling - https://en.wikipedia.org/wiki/Importance_sampling
- Rejection sampling - https://en.wikipedia.org/wiki/Rejection_sampling
- What is Rejection Sampling - <https://medium.com/data-science/what-is-rejection-sampling-1f6aff92330d>
- Sampling: Two Basic Algorithms - <https://gregorygundersen.com/blog/2019/09/01/sampling/>
- Masanori Hanada/MCMC-Sample-Codes - <https://github.com/masanorihanada/MCMC-Sample-Codes/tree/master>
- Estimating the value of Pi using Monte Carlo - <https://www.geeksforgeeks.org/dsa/estimating-value-pi-using-monte-carlo/>
- Pymc : Markov Chain Monte Carlo - https://www.pymc.io/projects/examples/en/latest/statistical_rethinking_lectures/08-Markov_Chain_Monte_Carlo.html
- Importance sampling - <https://github.com/kevinzakka/blog-code/blob/master/pr-Ir/Monte%20Carlo%20and%20Importance%20Sampling.ipynb>
- Rejection Sampling - <https://github.com/florianhartig/LearningBayes/blob/master/CommentedCode/02-Samplers/Rejection/ExampleRejectionSampler.md>